



Chuyên đề: Các Thuật toán Đường đi Ngắn nhất (Shortest Path Algorithms)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

🎯 1. Tổng quan & Bản đồ Tư duy (Mindmap)

Bài toán tìm **Đường đi ngắn nhất (Shortest Path)** là một trong những bài toán kinh điển và xuất hiện nhiều nhất trong các kỳ thi ICPC, Olympic Tin học Sinh viên và Codeforces. Việc lựa chọn sai thuật toán hoặc không nhận diện được tính chất của đồ thị (trọng số âm, đồ thị DAG, số lượng truy vấn) thường dẫn đến kết quả **TLE (Time Limit Exceeded)** hoặc **WA (Wrong Answer)**.

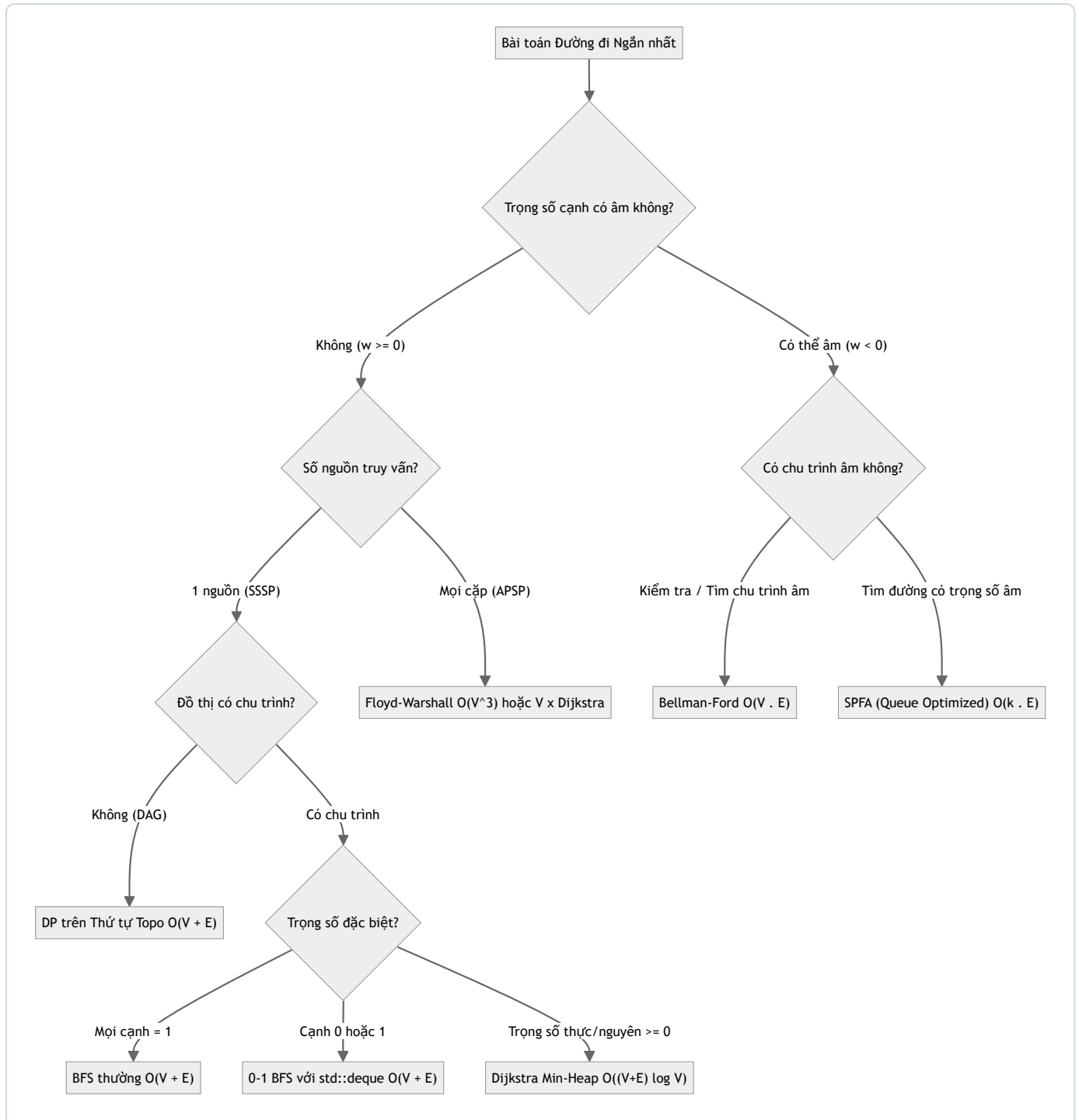
Thuật toán đường đi ngắn nhất được phân thành 2 lớp bài toán chính:

1. **SSSP (Single-Source Shortest Path)**: Tìm khoảng cách từ 1 đỉnh nguồn tới tất cả các đỉnh còn lại:

- Đồ thị không trọng số: **BFS** ($O(V + E)$).
- Trọng số 0 và 1: **0-1 BFS** với `std::deque` ($O(V + E)$).
- Đồ thị có hướng không chu trình (DAG): **Duyệt Topo** ($O(V + E)$).
- Trọng số không âm ($w \geq 0$): **Dijkstra** ($O((V + E) \log V)$).
- Có trọng số âm & phát hiện chu trình âm: **Bellman-Ford / SPFA** ($O(V \cdot E)$).

2. **APSP (All-Pairs Shortest Path)**: Tìm khoảng cách giữa mọi cặp đỉnh (u, v) :

- Đồ thị nhỏ ($V \leq 500$): **Floyd-Warshall** ($O(V^3)$).
- Đồ thị thưa trọng số không âm: Chạy **Dijkstra** V lần ($O(V \cdot E \log V)$).
- Đồ thị thưa có trọng số âm: **Thuật toán Johnson** ($O(V \cdot E \log V)$) kết hợp Bellman-Ford và tái đánh số trọng số).



⚡ 2. Thuật toán Dijkstra (Trọng số Không âm)

2.1. Bản chất Thuật toán & Bỏ đề Tham lam (Greedy)

Dijkstra duy trì một tập hợp các đỉnh đã xác định được khoảng cách ngắn nhất cuối cùng.

- Tại mỗi bước, thuật toán chọn ra đỉnh u chưa được chốt có giá trị $\text{dist}[u]$ **nhỏ nhất hiện tại**.
- **Bỏ đề tính đúng đắn:** Vì mọi trọng số cạnh đều **không âm** ($w \geq 0$), khoảng cách tới đỉnh u không bao giờ có thể được rút ngắn thêm qua bất kỳ đỉnh trung gian nào khác. Do đó, $\text{dist}[u]$ tại thời điểm lấy ra khỏi hàng đợi chính là khoảng cách ngắn nhất tối ưu tuyệt đối.

- Sau đó, ta tiến hành **thư giãn (Relaxation)** mọi cạnh đi ra từ u :

$$\text{Nếu } dist[u] + w(u, v) < dist[v] \implies dist[v] \leftarrow dist[u] + w(u, v)$$

2.2. Cài đặt C++ Chuẩn mực với Lazy Deletion

Trong thư viện STL C++, `std::priority_queue` không hỗ trợ thao tác `decrease_key`. Cách giải quyết chuẩn mực và đạt hiệu năng cao nhất trong CP là kỹ thuật **Lazy Deletion** (Đẩy bản sao mới vào heap và kiểm tra tính hợp lệ khi lấy ra):

```
const long long INF = 1e18; // Luôn dùng 1e18 để tránh tràn số

vector<long long> dijkstra(int s, int n, const vector<vector<pair<int, long long>>>& adj) {
    vector<long long> dist(n + 1, INF);
    // Min-Heap lưu cặp {khoảng cách, đỉnh}
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>>
    pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        // ⚠ DÒNG SỐNG CÒN: Bỏ qua các bản sao cũ đã bị tối ưu hóa trước đó
        if (d > dist[u]) continue;

        for (const auto& edge : adj[u]) {
            int v = edge.first;
            long long w = edge.second;

            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

⚠ Quy tắc vàng phòng thi:

Luôn có dòng `if (d > dist[u]) continue;`. Nếu thiếu dòng này, các bản sao lỗi thời sẽ tiếp tục duyệt các đỉnh kề của nó, khiến độ phức tạp thời gian thoái hóa thành cấp số mũ trên một số dạng đồ thị đối kháng (Anti-Dijkstra graphs), dẫn đến **TLE!**

2.3. Dijkstra Hai chiều (Bidirectional Dijkstra)

Khi cần tìm đường đi ngắn nhất giữa đúng 2 đỉnh S và T trên đồ thị lớn:

- Chạy đồng thời 2 lượt Dijkstra: Một lượt tiến từ S ($dist_S$) và một lượt lùi từ T trên đồ thị ngược ($dist_T$).
 - Thuật toán dừng lại khi một đỉnh u được lấy ra từ cả hai phía.
 - Khoảng cách ngắn nhất là $\min_x(dist_S[x] + dist_T[x])$ qua các đỉnh đã được thư giãn.
 - **Hiệu quả:** Không gian tìm kiếm giảm từ hình cầu bán kính R (πR^2) xuống 2 hình cầu bán kính $R/2$ ($2 \times \pi(R/2)^2 = \frac{1}{2}\pi R^2$), giảm số bước duyệt đi một nửa!
-

2.4. Tìm K Đường đi Ngắn nhất (K-Shortest Paths)

Thay vì chỉ lưu 1 khoảng cách nhỏ nhất, ta lưu một mảng 2 chiều hoặc danh sách:

- $dist[u][k]$: Khoảng cách ngắn thứ k từ S tới u ($1 \leq k \leq K$).
 - Mỗi đỉnh u có thể được lấy ra từ `priority_queue` tối đa K lần.
 - Độ phức tạp: $O(K \cdot (V + E) \log(V \cdot K))$.
-

3. Thuật toán Bellman-Ford & SPFA (Trọng số Âm)

3.1. Thuật toán Bellman-Ford & Nguyên lý Thư giãn Toàn cục

Bellman-Ford hoạt động dựa trên nguyên lý: Mọi đường đi đơn (không chứa chu trình) trên đồ thị V đỉnh đều có độ dài **tối đa $V - 1$ cạnh**.

- Do đó, nếu ta lặp lại thao tác thư giãn **toàn bộ E cạnh** của đồ thị đúng $V - 1$ lần, khoảng cách ngắn nhất tới mọi đỉnh chắc chắn được xác định chính xác.

```

bool bellman_ford(int s, int n, const vector<tuple<int, int, long long>>& edges, vector<long long>&
    dist.assign(n + 1, INF);
    dist[s] = 0;

    // Lặp n - 1 lần
    for (int i = 1; i <= n - 1; ++i) {
        bool any_update = false;
        for (const auto& [u, v, w] : edges) {
            if (dist[u] < INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                any_update = true;
            }
        }
        if (!any_update) break; // Tối ưu dừng sớm
    }

    // Lần lặp thứ n: Kiểm tra chu trình âm
    for (const auto& [u, v, w] : edges) {
        if (dist[u] < INF && dist[u] + w < dist[v]) {
            return true; // Tồn tại chu trình âm đến được từ s!
        }
    }
    return false;
}

```

3.2. Truy vết và In Chu trình Âm (Negative Cycle Detection & Trace)

Để tìm và in ra các đỉnh thuộc một chu trình âm:

1. Lưu vết đỉnh cha `parent[v] = u` mỗi khi thư giãn cạnh (u, v) .
2. Chạy V lần thư giãn. Nếu ở lần thứ V có đỉnh x vẫn bị giảm khoảng cách:
3. **Kỹ thuật nhảy lùi:** Cho x nhảy ngược V lần theo `parent`:

```
cpp for (int i = 0; i < n; ++i) x = parent[x];
```

Chứng minh: Vì chu trình có tối đa V đỉnh, nhảy V bước chắc chắn đảm bảo con trỏ x đã **lọt trọn vẹn vào bên trong chu trình âm!**

4. Lần theo `parent` từ x cho đến khi quay lại x để khôi phục toàn bộ chu trình.

3.3. Thuật toán SPFA (Shortest Path Faster Algorithm)

SPFA là bản nâng cấp hàng đợi của Bellman-Ford: Ta chỉ đẩy đỉnh v vào hàng đợi nếu khoảng cách `dist[v]` vừa được cập nhật giảm xuống.

- Duy trì mảng `in_queue[u]` (đỉnh đang nằm trong hàng đợi hay không) và `cnt[u]` (số lần đỉnh u được thư giãn).
- Nếu `cnt[u] >= n`, kết luận đồ thị chứa chu trình âm.
- **Độ phức tạp:** $O(k \cdot E)$ với $k \approx 2$ trên đồ thị ngẫu nhiên, nhưng có thể bị đẩy về $O(V \cdot E)$ trên đồ thị đặc biệt.

4. Thuật toán Floyd-Warshall (Mọi cặp Đỉnh - APSP)

4.1. Công thức Quy hoạch động

Gọi $dist[i][j]^{(k)}$ là độ dài đường đi ngắn nhất từ đỉnh i đến đỉnh j mà mọi đỉnh trung gian trên đường đi đều thuộc tập $\{1, 2, \dots, k\}$.

- Hệ thức truy hồi:

$$dist[i][j]^{(k)} = \min \left(dist[i][j]^{(k-1)}, dist[i][k]^{(k-1)} + dist[k][j]^{(k-1)} \right)$$

```
void floyd_warshall(int n, vector<vector<long long>>& dist) {  
    // ⚠️ QUY TẮC BẮT DI BẮT DỊCH: VÒNG LẶP k BẮT BUỘC PHẢI Ở NGOÀI CÙNG!  
    for (int k = 1; k <= n; ++k) {  
        for (int i = 1; i <= n; ++i) {  
            for (int j = 1; j <= n; ++j) {  
                if (dist[i][k] < INF && dist[k][j] < INF) {  
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);  
                }  
            }  
        }  
    }  
}
```

⚠️ Cạm bẫy cực kỳ phổ biến:

Nếu đặt vòng lặp `k` ở trong cùng (sau `i` và `j`), bạn sẽ chỉ tính được đường đi qua tối đa 1 đỉnh trung gian, dẫn đến sai hoàn toàn kết quả!

4.2. Nhận diện Chu trình Âm trong Floyd-Warshall

Sau khi chạy xong Floyd-Warshall:

- Đỉnh i nằm trong một chu trình âm khi và chỉ khi:

$$dist[i][i] < 0$$

- Đường đi giữa u và v có độ dài $-\infty$ nếu tồn tại một đỉnh k nằm trên chu trình âm sao cho $dist[u][k] < \infty$ và $dist[k][v] < \infty$.

5. Đồ thị Phân tầng (Layered Graph) & State Space Dijkstra

Trong CP, nhiều bài toán bổ sung thêm các điều kiện đặc biệt:

- "Được phép chọn tối đa K cạnh để miễn phí (trọng số thành 0)" (CSES Flight Discount).
- "Được phép đổi chiều tối đa K con đường".
- "Bình xăng có dung tích C , mỗi trạm có giá xăng khác nhau".

Kỹ thuật Nhân bản Trạng thái (Graph State Expansion):

Thay vì một đỉnh u , ta mở rộng không gian trạng thái thành cặp (u, k) :

- u : Đang ở đỉnh u ($1 \leq u \leq N$).

- k : Đã sử dụng k quyền ưu tiên ($0 \leq k \leq K$).

- **Các cạnh chuyển tiếp:**

1. Đi bình thường: $(u, k) \xrightarrow{w} (v, k)$.

2. Dùng quyền ưu tiên (nếu $k < K$): $(u, k) \xrightarrow{0} (v, k+1)$ (hoặc $\lfloor w/2 \rfloor$).

- Chạy thuật toán Dijkstra bình thường trên đồ thị có $N \cdot (K+1)$ trạng thái với khoảng cách `dist[u][k]`.

6. Đồ thị Đường đi Ngắn nhất (Shortest Path DAG)

Sau khi tính được mảng khoảng cách `dist` từ nguồn S bằng Dijkstra, ta có thể xây dựng **Đồ thị con các đường đi ngắn nhất (Shortest Path DAG)**:

- Cạnh có hướng (u, v) có trọng số w thuộc một đường đi ngắn nhất từ S khi và chỉ khi:

$$\text{dist}[u] + w == \text{dist}[v]$$

- **Tính chất kỳ diệu:** Đồ thị con gồm các cạnh thỏa mãn điều kiện trên **chắc chắn là một DAG** (Đồ thị có hướng không chu trình)!

- **Ứng dụng:**

1. **Đếm số lượng đường đi ngắn nhất:** Quy hoạch động trên DAG: $ways[v] = \sum_{(u,v) \in DAG} ways[u]$.

2. **Tìm cạnh cầu / cạnh xung yếu:** Cạnh mà mọi đường đi ngắn nhất đều bắt buộc phải đi qua.

7. Bảng So sánh Tổng hợp Thuật toán Shortest Path

Thuật toán	Loại bài toán	Trọng số áp dụng	Độ phức tạp Thời gian	Bộ nhớ	Khả năng phát hiện Chu trình âm
BFS	SSSP	Cạnh bằng nhau ($w = 1$)	$O(V + E)$	$O(V)$	Không áp dụng
0-1 BFS	SSSP	Cạnh $w \in \{0, 1\}$	$O(V + E)$	$O(V)$	Không áp dụng
Topo DP	SSSP	Bất kỳ trên DAG	$O(V + E)$	$O(V)$	Đồ thị không chu trình
Dijkstra	SSSP	Không âm ($w \geq 0$)	$O((V + E) \log V)$	$O(V + E)$	✗ Sai nếu có trọng số âm
Bellman-Ford	SSSP	Bất kỳ (kể cả âm)	$O(V \cdot E)$	$O(V + E)$	✓ Chính xác tuyệt đối
SPFA	SSSP	Bất kỳ (kể cả âm)	$O(k \cdot E)$ (TB)	$O(V + E)$	✓ Qua số lần vào queue $\geq V$

Thuật toán	Loại bài toán	Trọng số áp dụng	Độ phức tạp Thời gian	Bộ nhớ	Khả năng phát hiện Chu trình âm
Floyd-Warshall	APSP	Bất kỳ ($V \leq 500$)	$O(V^3)$	$O(V^2)$	✓ Khi $dist[i][i] < 0$

⚠ 8. Cạm bẫy Phòng thi & Sai lầm Thường gặp (Pitfalls)

1. **Tràn số 32-bit khi tính tổng khoảng cách:**
2. Trọng số cạnh $w \leq 10^9$ và đường đi có thể qua 10^5 cạnh $\implies$ Khoảng cách tối đa lên tới 10^{14} .
3. Khởi tạo khoảng cách bằng kiểu `int` hoặc `INF = 1e9` sẽ bị tràn số và dẫn đến sai kết quả. Bắt buộc dùng `long long` và `INF = 1e18` (hoặc `4e18`).
4. **Dùng Dijkstra trên đồ thị có trọng số âm:**
5. Dijkstra dựa vào tính chất tham lam đơn điệu. Nếu có cạnh âm, một đỉnh đã chốt khoảng cách vẫn có thể bị giảm tiếp, dẫn đến lập vô tận (TLE) hoặc kết quả sai (WA).
6. **Lỗi tràn bộ nhớ khi khởi tạo ma trận Floyd-Warshall:**
7. Ma trận $V \times V$ với $V = 10^5$ tốn ≈ 80 GB RAM $\implies$ Tràn bộ nhớ (MLE). Floyd-Warshall chỉ được dùng khi $V \leq 500$.
8. **Quên kiểm tra `dist[u] < INF` trước khi thư giãn:**
9. Trong Bellman-Ford và Floyd-Warshall: Nếu `dist[u] == INF` mà cộng thêm trọng số âm $w < 0$, giá trị sẽ bị giảm xuống $< INF$ giả tạo. Luôn kiểm tra `dist[u] < INF` trước khi cộng.

📖 9. Tài liệu Tham khảo & Đọc thêm

1. **Competitive Programmer's Handbook** — Antti Laaksonen:
2. Chapter 13: Shortest Paths (Dijkstra, Bellman-Ford, Floyd-Warshall).
3. **Introduction to Algorithms (CLRS 4th Edition):**
4. Chapter 22: Single-Source Shortest Paths.
5. Chapter 23: All-Pairs Shortest Paths.
6. **USACO Guide (Gold Section):**
7. [Shortest Paths with Non-Negative Edge Weights \(Dijkstra\)](#).
8. **VNOI Wiki:**
9. [Thuật toán Dijkstra](#).
10. [Thuật toán Bellman-Ford & SPFA](#).
11. [Thuật toán Floyd-Warshall](#).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.